

Sicurezza e Affidabilità

Safety engineering: mira a prevenire danni "catastrofici" causati dal cattivo funzionamento di un sistema informatico. Esempi di sistemi dove la safety è critica includono sistemi life-critical, sistemi avionici e controllori di centrali nucleari.

Security engineering: si occupa della prevenzione, difesa e recupero da danni contro la possibilità di attacchi (più o meno diretti) che possono mettere a repentaglio i valori rappresentati da un sistema informatico. Un principio fondamentale è che tutte le misure di safety sono anche misure di sicurezza, ma non è vero il viceversa.

Requisiti per la Sicurezza: la triade CIA

I requisiti fondamentali per la sicurezza sono noti come la triade CIA: Confidenzialità, Integrità e Disponibilità:

- **Confidenzialità (Confidentiality):** Assicura che l'informazione sia accessibile solo a chi è autorizzato. Si tratta della capacità di controllare e assicurare la riservatezza delle informazioni archiviate o trasmesse rispetto a soggetti non autorizzati.
- **Integrità (Integrity):** Assicura che l'informazione non sia stata modificata in modo non autorizzato. È la capacità di controllare e assicurare la non modificabilità e l'attendibilità delle informazioni memorizzate o trasmesse da parte di soggetti non autorizzati.
- **Disponibilità (Availability):** Assicura che gli utenti legittimi abbiano accesso all'informazione quando ne hanno bisogno. È la capacità di controllare e assicurare la possibilità di usare risorse e dei servizi agli utenti autorizzati.

Gli attacchi alla triade CIA sono tipicamente definiti come DAD:

- **Disclosure (Divulgazione)** contro la Confidenzialità.
- **Alteration (Alterazione)** contro l'Integrità.
- **Destruction (Distruzione)** contro la Disponibilità.

Ostacoli all'Implementazione della Sicurezza

Diversi fattori possono ostacolare l'implementazione della sicurezza:

- **Dilemma usabilità vs. sicurezza:** L'obiettivo dell'informatica è favorire l'accesso alle risorse, mentre la sicurezza obbliga a limitarlo. Ad esempio, l'autenticazione tramite password può essere vista come un fastidio da molti utenti.
- **Progetto tardivo:** Spesso la sicurezza viene considerata solo dopo il verificarsi dei primi incidenti.
- **Necessità di compromesso tra benefici e costi:** I sistemi di sicurezza hanno un costo e non sempre rientrano nel budget. Per questi motivi, una corretta analisi dei rischi diventa fondamentale.

Analisi e Gestione dei Rischi

L'analisi dei rischi è un processo cruciale. Il rischio è definito come la "possibilità di perdita o danno". In ambito di sicurezza informatica, il rischio (R) è una funzione di Vulnerabilità, Minacce e Valori.

$$R = \text{Vulnerabilità} \times \text{Minacce} \times \text{Valori}$$

- **Valori sensibili:** Identificano quali parti del sistema informativo sono importanti, sensibili o critiche. Si tratta degli elementi da proteggere e della loro relativa importanza.
- **Vulnerabilità:** Indica in cosa siamo vulnerabili, ovvero debolezze o attacchi che possono essere sfruttati per compromettere la confidenzialità, l'integrità o la disponibilità di valori sensibili.
- **Minacce:** Definiscono quali sono i pericoli cui siamo esposti, ovvero circostanze o agenti che possono arrecare danni.

La **gestione dei rischi** può essere visualizzata considerando la probabilità (data da vulnerabilità x minacce) e l'impatto rispetto ai valori sensibili. Questo approccio suggerisce diverse strategie:

- **Controllo e contenimento:** Per rischi ad alta probabilità e basso impatto.
- **Prevenzione e protezione:** Per rischi ad alta probabilità e alto impatto; qui si applicano principalmente i meccanismi di sicurezza.
- **Rischio ignorabile:** Per rischi a bassa probabilità e basso impatto.
- **Assicurazione, piano di backup, multiple point-of-failure:** Per rischi a bassa probabilità e alto impatto.

Crittografia

La crittografia è la disciplina che studia le tecniche per codificare/decodificare messaggi in modo da garantire la privacy della comunicazione fra due soggetti.

Un messaggio crittografico si compone di:

1. Insieme di messaggi in chiaro (plaintext)
2. Insieme di chiavi per codifica e decodifica
3. Funzione di codifica: $K \times P \rightarrow C$
4. Insieme di messaggi cifrati (cyphertext)
5. Funzione di decodifica: $K \times C \rightarrow P$

Vogliamo poter crittare un messaggio per proteggerlo dalle intrusioni, ma essere in grado di decodificarlo affinché il destinatario possa leggerlo.

Violazioni

Violazioni per forza bruta: una violazione per forza bruta consiste nel trovare tutte le istanze di un insieme finito di schemi crittografici che potrebbero corrispondere ad un messaggio, ad esempio provare tutte le possibili chiavi di N caratteri.

Crittografia e crittoanalisi

La crittografia si occupa degli Schemi, dei Metodi e degli Algoritmi per la codifica sicura dei messaggi.

La **crittoanalisi** è la disciplina che si occupa di **analizzare e cercare di violare sistemi crittografici**, cioè quei metodi usati per proteggere le informazioni attraverso la codifica dei messaggi (la crittografia).

Algoritmi degni di fiducia e utilizzabili

Un algoritmo crittografico è definito degno di fiducia e utilizzabile quando:

- È basato su concetti matematici consolidati.
- È stato analizzato da esperti.
- Ha superato la prova del tempo, ovvero non è mai stato violato.

Un algoritmo crittografico è definito utilizzabile quando:

- La complessità temporale dell'algoritmo per crittare / decrittare deve essere commisurata alla sicurezza necessaria.
- Le chiavi e il testo di input non devono essere soggetti a requisiti complessi.
- Il processo sottinteso dall'uso della crittografia deve essere semplice per l'utente.

Algoritmi crittografici

Esistono tre classi diverse di algoritmi crittografici:

1. **Algoritmi simmetrici** (detti anche a chiave segreta)
2. **Algoritmi asimmetrici** (detti anche a chiave pubblica)
3. **Algoritmi di hashing** (detti anche message digest)

Sistemi simmetrici (a chiave segreta)

Il funzionamento si basa sull'utilizzo della stessa chiave k sia per codifica che decodifica:

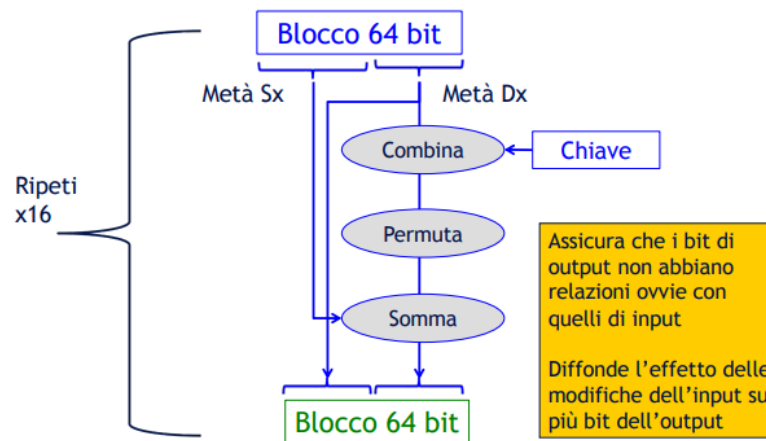
$$\text{decrypt}(k, \text{encrypt}(k, p)) = p$$

Per instaurare una comunicazione confidenziale, due soggetti devono conoscere una chiave k (non nota a nessun altro)

DES: Data Encryption Standard

Il Data Encryption Standard (DES) è un algoritmo di crittografia simmetrica a blocchi che opera su blocchi di 64 bit, utilizzando una chiave di 56 bit effettivi. Il processo di cifratura prevede una permutazione iniziale dei dati, seguita da 16 round di elaborazione secondo una struttura detta "rete di Feistel". In ciascun round, il blocco è diviso in due metà: la metà destra viene trasformata tramite una funzione complessa (funzione F) che include espansione, XOR con una sottochiave derivata dalla chiave principale, sostituzioni tramite S-box e permutazione; il risultato viene poi combinato con la metà sinistra tramite un'operazione XOR. Alla fine di ogni round, le due metà vengono scambiate.

Dopo l'ultimo round, le metà non vengono scambiate ma riunite e sottoposte a una permutazione finale. L'intero processo garantisce confusione e diffusione dei dati, rendendo difficile risalire al messaggio originale senza conoscere la chiave.



AES: Advanced Encryption Standard

L'AES (Advanced Encryption Standard) è un algoritmo di cifratura simmetrica a blocchi, progettato per essere sicuro ed efficiente. Opera su blocchi di 128 bit, trasformandoli attraverso una serie di passaggi ripetuti che includono sostituzioni di byte, scorrimenti delle righe e mescolamento delle colonne, combinati con l'applicazione di una chiave segreta. Le chiavi possono essere lunghe 128, 192 o 256 bit, e il numero di cicli (o round) di trasformazione varia di conseguenza: 10, 12 o 14 round. Grazie alla sua struttura modulare e all'efficienza delle sue operazioni, AES è adatto sia per implementazioni software che hardware, ed è considerato uno degli algoritmi di crittografia più affidabili e ampiamente utilizzati al mondo.

Supporto requisiti per sicurezza:

Requisito	Soddisfacimento	motivo
Confidenzialità	Sì	Solo chi conosce la chiave segreta può decodificare il messaggio
Integrità	Sì	Una volta che il messaggio è stato crittato, non è possibile manometterlo prima della ricezione (il messaggio decodificato è esattamente lo stesso messaggio che era stato crittato)
Autenticazione e non ripudio	Sì ma...	solo chi conosce la chiave segreta può essere mittente del messaggio

NB: Non è possibile garantire integrità e autenticazione in modo indipendentemente dalla confidenzialità (quindi non si può darne evidenza a terze parti... e quindi non adatto per firme digitali)

Quante chiavi segrete?

N individui per comunicare in maniera sicura a coppie hanno bisogno di $n(n - 1)/2$ chiavi, una per coppia.

Sistemi asimmetrici (a chiave pubblica)

Ogni soggetto possiede una coppia di chiavi:

- **Chiave pubblica:** è conosciuta da tutti
- **Chiave privata:** non è condivisa con nessuno

Un messaggio codificato con una delle chiavi può essere decodificato solo con la chiave corrispondente:

$$\text{decrypt}(k_{\text{priv}}, \text{encrypt}(k_{\text{pub}}, p)) = p$$

$$\text{decrypt}(k_{\text{pub}}, \text{encrypt}(k_{\text{priv}}, p)) = p$$

Quante chiavi segrete?

Per N individui c'è bisogno di N coppie di chiavi

Supporto requisiti per sicurezza:

Requisito	Soddisfacimento	motivo
Confidenzialità	Sì	Solo chi conosce la chiave segreta può decodificare il messaggio crittato con la corrispondente chiave pubblica
Integrità	Sì	una volta che il messaggio è stato crittato, con chiave pubblica o privata, non è possibile manometterlo prima della decodifica
Autenticazione e non ripudio	Sì	Solo chi conosce la chiave segreta può essere il mittente di un messaggio che si decodifica con la corrispondente chiave pubblica

RSA

RSA è un algoritmo di crittografia asimmetrica che utilizza una coppia di chiavi: una pubblica e una privata, calcolate a partire da due numeri primi molto grandi scelti casualmente. I numeri primi vengono scartati dopo la generazione delle chiavi e non devono essere rivelati. La sicurezza dell'algoritmo si basa sulla difficoltà di fattorizzare numeri molto grandi.

La chiave pubblica viene usata per cifrare i messaggi, mentre quella privata per decifrarli. Viceversa, per le firme digitali si cifra con la chiave privata e si verifica con quella pubblica.

Hashing (Message Digest)

L'hashing, o message digest, è un processo crittografico che trasforma un messaggio di qualsiasi lunghezza in una stringa di lunghezza fissa chiamata digest o codice riassuntivo. Questa stringa rappresenta in modo univoco (idealmente) il contenuto del messaggio originale. Le funzioni di hash crittografiche sono progettate per essere veloci da calcolare, ma impossibili da invertire: dato un digest, non è possibile risalire al messaggio iniziale. Inoltre, devono essere resistenti alle collisioni, cioè deve essere estremamente improbabile che due messaggi diversi producano lo stesso digest. Il message digest viene utilizzato principalmente per verificare l'integrità dei dati: se due messaggi producono lo stesso digest, si può presumere che il contenuto non sia stato alterato.

Algoritmi di hashing

Esistono diversi algoritmi di hashing:

- MD5 (hash di 128 bit)
- SHA1 (hash di 160 bit)
- SHA2 (256, 384, 512)

Message Digest Algorithm 5

L'algoritmo MD5 è una funzione di hash crittografica che prende in input un messaggio di lunghezza arbitraria e produce un'impronta digitale (digest) di 128 bit. Il messaggio viene prima pre-elaborato tramite un padding che lo rende un multiplo di 512 bit, poi suddiviso in blocchi da 512 bit ciascuno. A partire da un valore iniziale fisso (riassunto iniziale), ogni blocco viene elaborato attraverso una funzione di compressione che aggiorna progressivamente lo stato dell'hash. Alla fine dell'elaborazione di tutti i blocchi, il valore risultante rappresenta l'hash MD5 del messaggio. Questo processo garantisce che anche una minima variazione nel messaggio produca un hash completamente diverso, grazie all'effetto valanga.

Requisito	Soddisfacimento	Motivo
Confidenzialità	No	Nessuno può decodificare il codice MD di un messaggio (non invertibilità)
Integrità	Si	La coppia permette di stabilire che il messaggio non sia stato manomesso
Autenticazione e non ripudio	No	Chiunque può aver generato il codice MD di un dato messaggio

Prestazione degli algoritmi

Gli algoritmi DES e MD5 offrono prestazioni significativamente superiori rispetto a RSA, essendo diversi ordini di grandezza più veloci. In particolare, quando sono implementati in hardware dedicato (come un chip), DES e MD5 possono raggiungere velocità dell'ordine dei gigabit al secondo, mentre RSA si limita a velocità dell'ordine dei kilobit al secondo. Questo rende DES e MD5 molto più adatti per l'elaborazione di grandi quantità di dati, mentre RSA viene tipicamente utilizzato per cifrare solo piccole quantità di informazioni, come ad esempio una chiave di sessione. In ambito pratico, si combinano RSA e DES per ottenere sia sicurezza che performance: RSA viene usato per proteggere la chiave di sessione, mentre DES si occupa della cifratura del messaggio vero e proprio.

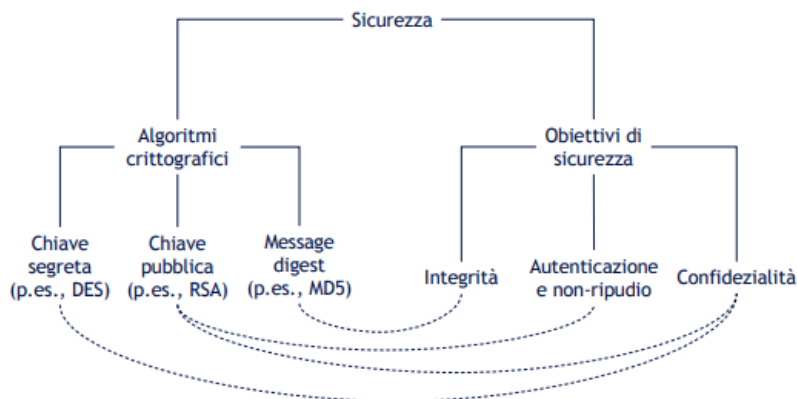
Efficienza nelle firme digitali

Le firme digitali sono uno strumento fondamentale per garantire l'integrità e l'autenticità dei messaggi, ma firmare un messaggio di grandi dimensioni (ad esempio con RSA) può risultare poco efficiente. Per ovviare a questo problema, si firma solo una piccola parte del messaggio, il message digest, ovvero il riassunto del contenuto ottenuto tramite una funzione di hash. Si calcola il digest del messaggio, M = messaggio, $F = \text{MessageDigest}(M)$, $F' = \text{encrypt}(F, k_{\text{priv}})$. Il destinatario riceve il messaggio in chiaro e la firma cifrata F' , la decifra con la chiave pubblica del mittente per ottenere il digest originale, e lo confronta con il digest calcolato localmente sul messaggio ricevuto. Se i due digest coincidono, il messaggio è integro e proviene effettivamente dal mittente. Questo metodo permette di ridurre il carico computazionale senza compromettere la sicurezza della firma.

Certificati X.509

Un certificato X.509 è un documento digitale che garantisce la veridicità di una chiave pubblica associata a una determinata entità, come una persona o un'azienda. Questo certificato viene emesso da un'autorità di certificazione ritenuta fidata e autorizzata. Lo standard X.509 definisce il formato di tali certificati, che contengono informazioni fondamentali come il nome dell'entità, la sua chiave pubblica, il nome dell'autorità di certificazione che lo ha rilasciato e la firma digitale di quest'ultima. Se si dispone della chiave pubblica dell'autorità di certificazione, è possibile verificarne la firma e quindi l'integrità e l'autenticità del certificato. In questo modo, si può essere certi che la chiave pubblica riportata nel certificato appartenga realmente all'entità indicata.

Riassunto



Sicurezza nei sistemi operativi

Le aree di interesse per la sicurezza nei sistemi operativi riguardano:

- Autenticazione degli utenti
- Protezione della memoria
- Controllo degli accessi
 - File
 - Periferiche di I/O
 - Oggetti generici
- Controllo della condivisione fra utenti
- Protezione dei dati di protezione del sistema operativo

Schemi di autenticazione

L'obiettivo dell'autenticazione è quello di impedire l'accesso di soggetti non autorizzati alle risorse del sistema.

Esistono diversi metodi per l'autenticazione:

- Dimostrazione di conoscenza: PIN – password
- Dimostrazione di possesso: chiavi fisiche – smartcard di identificazione
- Caratteristiche personali uniche: impronte digitali – riconoscimento del viso

Password file criptati

Per proteggere le password degli utenti, si utilizzano tecniche di crittografia unidirezionale come MD5 o SHA-2, che permettono di memorizzare un hash della password invece della password in chiaro. Per evitare che utenti con la stessa password ottengano lo stesso hash, si utilizza un valore casuale chiamato "salt", tipicamente di 12 bit. Il salt viene salvato insieme all'hash nel file delle password, e la password viene memorizzata come hash (password + salt). Questa tecnica aumenta la sicurezza del sistema, ma è comunque fondamentale che il file delle password sia protetto anche dal sistema operativo, per evitare accessi non autorizzati.

Processo di autenticazione

Un sistema di autenticazione ben progettato ha le seguenti caratteristiche:

- Non fornisce dettagli su nomi utente e sistema prima che l'autenticazione sia avvenuta con successo.
- Può essere lento per scoraggiare attacchi di forza bruta.
- Può disabilitare gli utenti dopo alcuni tentativi falliti.
- Può utilizzare un meccanismo di autenticazione a più fattori.
- Deve essere resistente ad attacchi basati sul tempo di risposta.

Autenticazione Challenge/Response

Nell'autenticazione challenge/response (o one-time password), la password è una funzione matematica $F(x)$ che l'utente sa calcolare.

- Il sistema propone un valore casuale (X) (challenge).
- Il richiedente deve rispondere con il valore corretto $F(X)$ (response). La funzione $F(x)$ può essere una funzione "segreta" di numeri o stringhe, un generatore di numeri condiviso, o una funzione crittografica.

Quando l'autenticazione avviene su una rete (es. Internet), le credenziali (come username e password) possono essere intercettate da un **"Man-in-the-middle"**, che potrebbe poi impersonare l'utente legittimo. Per rendere sicuri i protocolli di autenticazione challenge/response si usa la crittografia, si cambia il challenge ad ogni tentativo (usando un **"nonce"**, ovvero un messaggio usato una sola volta) e si inseriscono identificativi nei messaggi criptati.

Un esempio di autenticazione challenge/response con crittografia prevede che client e server condividano una chiave segreta K .

1. Il client inizia l'autenticazione.
2. Il Server invia un challenge (es. "encrypt something").
3. Il client risponde con $\text{encr}_K(\text{yes})$.
4. Il Server decifra il messaggio e, se è "yes", autentica il client.

Tuttavia, se un malintenzionato intercetta il messaggio cifrato trasmesso dal client ($\text{encr}_K(\text{yes})$) potrebbe riutilizzarlo per farsi riconoscere dal server come client. Per risolvere questo problema, il server invia un "nonce" (un valore usato una sola volta) come challenge. Il client cifra il nonce con la chiave K . Se il malintenzionato intercetta $\text{encr}_K(\text{Nonce})$ e prova a riutilizzarlo, il server, aspettandosi un nonce diverso (N_2), non lo autenticherà.

Spesso l'autenticazione è bidirezionale: non solo il client si autentica al server, ma anche il server si autentica al client. In questo caso, il client invia $\text{encr}_K(N_s)$ dove N_s è un nonce inviato dal server e un suo nonce N_A . Il server risponde con $\text{encr}_K(N_A)$.

Reflection attack e soluzioni

Un **reflection attack** può verificarsi se l'autenticazione bidirezionale usa lo stesso protocollo in entrambe le direzioni. Il malintenzionato volendo impersonare il client verso il server, potrebbe iniziare una richiesta di autenticazione al server. Il server risponde con un nonce N . Il malintenzionato non conoscendo la chiave K , non può calcolare $\text{encr}_K(N)$. Però può avviare parallelamente un'autenticazione verso il client (fingendosi il server), inviando N come challenge. Il client risponderà al malintenzionato con $\text{encr}_K(N)$, che il malintenzionato potrà poi inoltrare al server originale per impersonare il client.

La soluzione a questo problema è eliminare la simmetria aggiungendo l'identificativo del mittente / destinatario nei messaggi cifrati. Ad esempio, quando il server risponde al challenge N del client invierà $\text{encr}_K(\text{server}, N)$. Se il malintenzionato inoltra questo messaggio come risposta al server, il

server, decifrandolo, verificherà che l'identificativo ("server") non corrisponde a quello che si aspetta da un client.

Protocollo Needham-Schroeder

Il protocollo Needham-Schroeder per l'autenticazione bidirezionale asimmetrica (con chiave pubblica) prevede i seguenti passaggi:

1. Il client invia al Server: $encr_{PK-S}(A, NA)$ (identità del client e un nonce NA, cifrati con la chiave pubblica del Server). Questo è il challenge per il server.
2. Il Server risponde al client: $encr_{PK-A}(A, NA)$ (il nonce del client NA e un nuovo nonce del Server NS, cifrati con la chiave pubblica del client). Questa è la conferma e un challenge per il client.
3. Il client risponde al Server: $encr_{PK-S}(NS)$ (il nonce del Server NS, cifrato con la chiave pubblica del Server).

Questo protocollo mira a soddisfare i tre requisiti: uso di crittografia con challenge-response, nonce variabili, e inserimento di identificativi. Tuttavia, la versione originale del protocollo è vulnerabile perché il secondo (e anche il terzo) messaggio non include l'identità del mittente (il Server). Questo bug è stato identificato 17 anni dopo l'ideazione del protocollo.

Controllo degli accessi agli oggetti protetti

Il controllo degli accessi si riferisce ai meccanismi che regolano l'accesso alle risorse protette all'interno di un sistema. È importante notare che gli individui autorizzati ad accedere al sistema non possiedono diritti omogenei per tutte le risorse.

Password di accesso alle risorse

L'uso di password per accedere a risorse specifiche è un possibile meccanismo per restringere i diritti di accesso. Tuttavia, questo approccio presenta diversi svantaggi:

- Limitato alla scelta di accesso / non accesso.
- Introduce una password per ogni risorsa.
- Difficoltà nel tracciare gli utenti che hanno accesso a determinate risorse.
- Difficoltà nella gestione di programmi che accedono a molte risorse
- Difficoltà nella revoca dei diritti.

Access control list

La **Access Control List** è un meccanismo avanzato di gestione dei permessi che permette di specificare, per ogni risorsa (come un file o una cartella), quali utenti o gruppi possono accedervi e con quali diritti. Ogni voce della lista associa un utente o gruppo a uno o più permessi, come lettura (R), scrittura (W), esecuzione (X) e proprietà (O). Ad esempio, è possibile consentire l'accesso in lettura e scrittura a un utente specifico, mentre a un intero gruppo si può garantire solo il permesso di lettura. Le ACL offrono un controllo molto dettagliato e sono particolarmente utili in ambienti in cui

più utenti condividono le stesse risorse. Tra i vantaggi principali vi sono la facilità nel determinare chi ha accesso a cosa e la semplicità nella modifica o revoca dei diritti di accesso.

Bit di protezione

Il sistema dei **bit di protezione**, utilizzato nei sistemi Unix/Linux tradizionali, rappresenta una versione semplificata delle ACL. In questo modello, ogni file o directory ha associati 9 bit che definiscono i permessi per tre categorie: il proprietario (OWNER), il gruppo (GROUP) e tutti gli altri utenti (WORLD). Ogni categoria dispone di tre permessi: lettura (R), scrittura (W) ed esecuzione (X), rappresentati in binario come 0 (permesso negato) o 1 (permesso concesso).

Obiettivi generali del controllo degli accessi

Gli **obiettivi generali del controllo degli accessi** includono innanzitutto la **mediazione completa**, cioè la necessità di controllare ogni accesso alle risorse per prevenire attacchi come il TOC/TOU (Time-of-Check/Time-of-Use), dove le condizioni di sicurezza cambiano tra il momento del controllo e quello dell'uso effettivo della risorsa. Un altro obiettivo fondamentale è l'attuazione del principio dei **privilegi minimi**, che prevede che ogni utente o processo debba disporre solo dei permessi strettamente necessari alle proprie funzioni. Infine, è essenziale **controllare l'uso appropriato degli** oggetti protetti, evitando anche perdite indirette di informazioni (leak) verso soggetti non autorizzati. Questi principi sono alla base della sicurezza nei sistemi informatici moderni.

Covert channels

I **covert channels** sono modalità di trasmissione di dati sensibili attraverso canali alternativi e condivisi, spesso legittimi, ma utilizzati in modo improprio da soggetti non autorizzati. Questi canali possono sfruttare segnali “non ovvi” per eludere i controlli di sicurezza. Un tipico esempio è rappresentato da un attaccante interno (insider) o un Trojan che opera tramite un programma di servizio. I meccanismi impiegati includono la scrittura di dati su file con livelli di protezione diversi, l'uso di nomi di file per codificare informazioni, oppure l'introduzione di codifiche nei file legali attraverso dettagli apparentemente insignificanti, come il numero di spazi doppi o l'ultima cifra non significativa di un campo. Altri metodi prevedono la segnalazione di eventi osservabili, come la presenza o assenza di oggetti in memoria o la modifica dello stato di file in momenti prestabiliti, per comunicare informazioni in modo nascosto.

Tipologie di controllo degli accessi

- **Discretionary Access Control (DAC):** (es. Access Control List) Il proprietario di una risorsa può concedere o modificare l'accesso e i diritti ad altri a sua discrezione.
- **Mandatory Access Control (MAC):** Il sistema impone un modello che limita e controlla la discrezionalità degli utenti nell'assegnare i diritti di accesso alle risorse.

Modelli di sicurezza MAC

I modelli di sicurezza MAC definiscono in maniera precisa la relazione di accessibilità tra soggetti (elementi attivi come utenti, processi) e oggetti (risorse contenenti dati sensibili come file, memoria) del sistema. Questi modelli sono basati su obiettivi e requisiti di sicurezza specifici, fortemente

dipendenti dal dominio applicativo. Un principio fondamentale è che se un soggetto rilascia un oggetto sensibile, bisogna gestire opportunamente i casi in cui la risorsa viene acquisita da un altro soggetto.

Sicurezza multi-livello: confidenzialità

Un primo esempio di modello MAC è la sicurezza multi-livello, nata in ambienti militari, dove la confidenzialità dei dati è spesso il requisito principale. Questa politica classifica i livelli di sicurezza per soggetti e oggetti, e l'accesso è consentito solo se il livello del soggetto è maggiore o uguale al livello dell'oggetto (livello soggetto $\geq$ livello oggetto).

Modello di Bell – LaPadula

Il modello di **Bell-LaPadula** è un modello di controllo accessi MAC orientato alla confidenzialità che garantisce due proprietà:

- **Simple security property (no-read-up):** Un soggetto non può leggere oggetti di classificazione più alta.
- **Confinement property (no-write-down):** Un soggetto non può scrivere su oggetti di classificazione più bassa

Modello BIBA

Il modello BIBA è un modello di controllo accessi MAC orientato all'integrità che garantisce due proprietà:

- **Simple integrity axiom (no-write-up):** Un soggetto non può modificare oggetti di classificazione più alta.
- **Integrity confinement axiom (no-read-down):** Un soggetto non può leggere oggetti di classificazione più bassa.

Modelli Multi-Laterali

A differenza dei modelli di sicurezza multi-livello che controllano i flussi di informazione in base al livello di criticità, i modelli di sicurezza multi-laterali controllano i flussi di informazione in base a raggruppamenti semantici dei dati e degli utenti (es. compartimenti, progetti, gruppi, ruoli).

Modello Bell-LaPadula esteso con compartimenti

Una estensione del modello Bell-LaPadula, originata dalla policy di sicurezza militare, associa informazioni e soggetti a "compartimenti" (progetto, missione, ecc.). La regola della "necessità di conoscere" stabilisce che l'accesso è consentito solo se: livello soggetto $\geq$ livello oggetto E i compartimenti dell'oggetto sono un sottoinsieme dei compartimenti del soggetto (compartimenti oggetto $\subseteq$ compartimenti soggetto).

Muraglia cinese

Il modello della Muraglia Cinese ha l'obiettivo di gestire i conflitti di interesse (COI), controllando il flusso di dati critici tra aziende concorrenti. Limita l'accesso alle informazioni riservate di una compagnia da parte di consulenti. La politica di accesso organizza i dati su tre livelli:

1. Oggetti singoli.
2. Dataset aziendali (ogni oggetto in un unico dataset).
3. Classi di COI fra dataset (ogni dataset in un'unica classe). La regola di accesso per un soggetto S e un oggetto O è: S può accedere a O solo se O appartiene allo stesso dataset di un oggetto O' a cui S ha già avuto accesso oppure il dataset di O non appartiene alla stessa classe di conflitto di interesse del dataset di O'.

Auditing e intrusion detection

L'auditing implica il tracciamento da parte del Sistema Operativo di varie attività, tra cui:

- Lettura e scrittura di dati (accesso a file, memoria, ecc.).
- Creazione o distruzione di un oggetto.
- Accesso degli utenti al sistema (login e logout).
- Operazioni amministrative (creazione di utenti, ecc.).
- Operazioni in modalità privilegiata.
- Operazioni non usuali.

Gli **Intrusion Detection Systems (IDS)** sono sistemi (automatici e non) che analizzano i dati tracciati (audit trails) per determinare o prevenire tentativi di compromissione o di uso non autorizzato di un sistema.

SO Trusted: Principi di Progettazione

I principi di progettazione per un Sistema Operativo Trusted (affidabile) includono:

- Minimi privilegi.
- Mediazione completa.
- Autorizzazione negata per default.
- Evitare "Multiple point of failure" (Singoli punti di guasto critici).
- Separazione fra utenti, Minima condivisione.
- Protezione rispetto al riutilizzo delle risorse.
- Accounting and audit (Contabilità e verifica).

Questi principi devono essere considerati tenendo conto dei requisiti di usabilità:

- Economia del meccanismo.
- Struttura aperta (basata sulla segretezza di pochi elementi critici).
- Facilità di uso.

Sicurezza nelle reti

Obiettivi e Vulnerabilità della Sicurezza di Rete

La sicurezza nelle reti di computer condivide gli stessi obiettivi fondamentali della sicurezza di un singolo host: confidenzialità, integrità, disponibilità, controllo degli accessi e autenticazione/non ripudio. Tuttavia, la connessione a una rete, come Internet, espone un computer a un numero molto maggiore di potenziali attacchi, poiché diventa accessibile a milioni di altri sistemi.

I messaggi scambiati attraversano nodi intermedi non controllabili e l'automazione degli attacchi è resa più semplice dal software di comunicazione. Le principali violazioni della sicurezza includono l'**intercettazione** (che mina la confidenzialità), la **modifica o creazione di traffico** (che compromettono l'integrità) e il **denial of service** (che pregiudica la disponibilità).

Sniffing

Uno sniffer è un software utilizzato per monitorare e analizzare il traffico di rete. La sua funzione principale è quella di intercettare e acquisire i pacchetti di dati che transitano su una rete, operando al livello datalink. Normalmente, una scheda di rete riceve solo i pacchetti indirizzati al proprio MAC address. Tuttavia, quando viene impostata in modalità promiscua, essa è in grado di catturare tutti i pacchetti che circolano nella rete locale, indipendentemente dal destinatario. Questo permette allo sniffer di ottenere una copia di ogni pacchetto in transito. È in grado di interpretare e visualizzare in chiaro le informazioni contenute negli header dei pacchetti di livello 2, 3 e 4, oltre ai protocolli di livello applicazione come FTP, HTTP, ecc.

Applicazioni dello sniffing

- **Analisi automatica della rete:** si avvale di strumenti e software capaci di esaminare il traffico in modo sistematico, ad esempio attraverso la ricerca di pattern specifici, come password e nomi utente trasmessi in chiaro. Questa tecnica è spesso sfruttata da hacker e cracker per intercettare credenziali sensibili e ottenere accesso non autorizzato a sistemi vulnerabili.
- **Analisi delle anomalie:** serve a individuare problemi all'interno della rete, come ad esempio capire perché il computer A non riesce a comunicare con il computer B. Questo tipo di analisi è utile per diagnosticare malfunzionamenti o comportamenti anomali.
- **Analisi delle prestazioni:** utilizzata per identificare colli di bottiglia o rallentamenti nel funzionamento della rete.
- **Monitoraggio della rete (difensivo):** consente di rilevare attacchi o minacce in tempo reale, nonché attività sospette o malevole in corso.
- **Registrazione del traffico di rete:** tutte le transazioni di rete possono essere registrate in log, così da poter essere analizzate successivamente in caso di incidenti.

Nelle reti commutate, il traffico viene trasmesso in base all'associazione tra indirizzo MAC e porta dello switch, escludendo i terminali non destinatari del traffico. Di conseguenza, uno sniffer può intercettare solo il traffico destinato al proprio host, ma esistono tecniche che permettono comunque di eseguire sniffing anche su reti commutate.

- Configurare la porta dello switch a cui è collegato lo sniffer in modalità mirroring. In questa modalità, lo switch replica il traffico ricevuto da qualsiasi porta sulla porta dello sniffer, permettendogli così di osservare tutto il traffico della rete.

Se la funzionalità di port mirroring non è disponibile, esistono alcune alternative per eseguire lo sniffing su reti commutate:

- Utilizzare dispositivi **repeater** (con banda limitata): replicano il traffico su tutte le porte, permettendo l'intercettazione, ma sono adatti solo a reti semplici o a bassa velocità.
- Usare sonde hardware dedicate (TAP — Test Access Point): questi dispositivi si inseriscono fisicamente tra due nodi di rete e copiano tutto il traffico in transito verso una terza porta per l'analisi.
- Deviare il traffico tramite attacchi mirati, ad esempio: ARP Poisoning (o ARP spoofing): consiste nel falsificare le tabelle ARP dei dispositivi in rete per dirottare il traffico verso l'attaccante, che può così intercettarlo prima di inoltrarlo alla destinazione.

ARP Poisoning

L'ARP poisoning sfrutta la natura stateless del protocollo ARP, ovvero la capacità di interpretare ogni pacchetto in modo isolato, basandosi unicamente sui dati che contiene, senza verifiche sull'origine. Le risposte ARP, infatti, vengono accettate e memorizzate nella cache anche se non sono state richieste (assenza di controllo sul mittente). Un attaccante può quindi inviare una risposta ARP contraffatta a un host o a un router, facendo sembrare che il proprio indirizzo MAC corrisponda a un determinato indirizzo IP. In questo modo, il traffico destinato a quell'IP viene deviato verso l'attaccante, che può intercettarlo, analizzarlo e, per non farsi notare, inoltrarlo alla reale destinazione. Poiché le voci nella cache ARP hanno una durata limitata, l'attacco deve essere periodicamente rinnovato con nuove risposte falsificate. Questo tipo di attacco è possibile perché i protocolli stateless consentono di impostare liberamente gli indirizzi di origine: le vittime accettano i pacchetti così come sono, assumendo che provengano realmente dall'indirizzo indicato nel pacchetto.

Difese contro l'ARP poisoning

- Voci ARP statiche: è possibile configurare manualmente una tabella ARP “fissa” all'interno dei dispositivi, associando un indirizzo IP specifico a un indirizzo MAC.
- Software di rilevamento e prevenzione: esistono strumenti in grado di analizzare il traffico ARP per individuare comportamenti sospetti.
- Sicurezza a livello di sistema operativo: alcuni sistemi operativi includono contromisure integrate contro gli attacchi ARP. Alcuni ignorano le risposte ARP non richieste, evitando di aggiornare la cache con dati non sollecitati.

Wiretap passivo

Intercettazione dei pacchetti di rete tramite sniffing, cioè l'analisi del traffico che transita sulla rete. Può avvenire anche tramite:

- Access point wireless non protetti o mal configurati.
- Mezzi fisici, come la lettura per induzione elettromagnetica, la manomissione dei cavi o l'intercettazione di comunicazioni wireless o satellitari tramite antenne specializzate.

L'analisi del traffico non si limita ai dati trasmessi, ma include anche i metadati della comunicazione, come gli indirizzi di origine e destinazione, che possono fornire informazioni sensibili.

IP spoofing

Tecnica in cui un attaccante manipola i pacchetti IP in modo che sembrino provenire da un indirizzo IP diverso da quello reale. In particolare, l'indirizzo IP sorgente viene falsificato per nascondere l'identità dell'attaccante o per aggirare meccanismi di sicurezza basati sull'identificazione dell'IP.

Pharming

Tecnica di spoofing in cui un host malevolo si finge un altro host legittimo, solitamente per rubare dati riservati (come password, codici bancari, informazioni personali). L'obiettivo è indurre l'utente a inserire i propri dati in una copia falsa di un sito di fiducia, senza accorgersi dell'inganno. L'attaccante:

- Cerca di replicare esattamente l'aspetto del sito originale, imitando layout, logo, e testi.
- Sfrutta una confusione fra URL con nomi simili (es. g00g1e.com invece di google.com) in modo che l'utente venga indirizzato a un sito truffaldino.
- Manomette le tabelle DNS, in modo che, anche digitando l'URL corretto, il browser venga reindirizzato a un IP falso, che ospita il sito truffa.
- Invia e-mail o messaggi ingannevoli contenenti link a siti camuffati. L'utente clicca e finisce su un sito graficamente identico a quello reale, dove inserisce inconsapevolmente i propri dati.

Phishing

Tecnica di ingegneria sociale usata dagli attaccanti per ingannare le persone e spingerle a fornire informazioni sensibili. L'attaccante invia messaggi falsi simili a quelli ufficiali di banche, aziende, servizi online con:

- Un linguaggio allarmante o urgente
- Un link ingannevole che porta a una pagina web falsa (ma simile a quella vera)
- Richiesta di inserire i dati personali, che vengono poi salvati dall'attaccante

Spoofing: Fake Public Key Identity

Attacco in cui un attaccante finge di essere un altro utente o server fornendo una falsa chiave pubblica spacciata per quella legittima. L'obiettivo è ingannare il destinatario, facendogli crittografare dati sensibili con la chiave sbagliata, che in realtà è controllata dall'attaccante.

- L'attaccante si spaccia per un server o un utente fidato.
- Fornisce una propria chiave pubblica, fingendo che appartenga a qualcun altro

- La vittima, non accorgendosi dell'inganno, crittografa i dati con quella chiave.
- L'attaccante riceve i dati cifrati, li decifra con la propria chiave privata e li legge in chiaro.

Spoofing: Fake Public Key Identity con Man in the Middle

L'utente A vuole comunicare in modo sicuro con B, usando la crittografia a chiave pubblica. L'attaccante M si interpone tra A e B, fingendosi l'altro interlocutore per ciascuno di essi

- A richiede la chiave pubblica di B.
- M intercetta la richiesta e invia la propria chiave pubblica a B, fingendosi A.
- B cripta il messaggio con la chiave ricevuta (che in realtà è di M) e lo invia.
- M riceve il messaggio, lo decifra con la sua chiave privata, legge o modifica il contenuto
- M cripta di nuovo il messaggio con la vera chiave pubblica di A e lo invia ad A.

M, essendo in possesso della chiave di sessione, può leggere/modificare tutti i messaggi cifrati che passano tra A e B, continuando a ritradurre ogni comunicazione da uno all'altro.

Denial of Service (DoS)

Mira a rendere un servizio o una rete non disponibile agli utenti legittimi, sovraccaricando risorse come CPU, RAM, banda o numero di connessioni disponibili.

Meccanismo di three-way handshake

Per stabilire una connessione TCP, viene utilizzato un meccanismo chiamato three-way handshake, che funziona nel seguente modo:

- Il client invia un pacchetto SYN al server per avviare la connessione.
- Il server risponde con un pacchetto SYN-ACK.
- Il client completa il processo rispondendo con un pacchetto ACK. A questo punto, la connessione è stabilita.

SYN Flood

Il SYN Flood è un tipo di attacco DoS che sfrutta proprio il meccanismo di handshake TCP:

- L'attaccante invia una grande quantità di pacchetti SYN verso il server.
- Ogni pacchetto contiene un indirizzo IP falso (spoofed).
- Il server risponde con SYN-ACK a quegli IP, ma non riceve mai l'ACK finale, perché l'indirizzo è falso.
- Il server mantiene la connessione "in attesa" in una coda, consumando risorse.
- Dopo un certo numero di richieste, la coda si riempie e il server non può più accettare nuove connessioni legittime.

Principali categorie di DoS

- Attacchi fisici: colpiscono direttamente hardware (computer, router, switch, cavi, ecc.).
- Flooding (sovraccarico di traffico)
 - SYN Flooding (su TCP/UDP)

- ICMP Flooding: sovraccarico tramite pacchetti ICMP (es. ping) per consumare banda risorse CPU.
- Smurf Attack: invia un ping ICMP a un indirizzo broadcast della rete, con l'IP sorgente falsificato impostato sull'obiettivo. Tutti gli host nella rete rispondono al bersaglio, generando un sovraccarico
- **Attacchi DDoS (Distributed DoS):** Simile al DoS, ma distribuito su più dispositivi compromessi (zombie/bot).
- I dispositivi infatti (tramite cavalli di Troia, malware) vengono controllati da remoto per lanciare attacchi coordinati e molto più dannosi.

Difese contro attacchi dalla rete

Security Enforcement Devices

I router di confine e i firewall sono dispositivi hardware distinti. Tuttavia, molte delle funzionalità di un firewall possono essere implementate direttamente in un router di confine.

Firewall

Letteralmente significa “barriera isolante dal fuoco”. È il principale componente passivo di difesa perimetrale. Svolge compiti di enforcement della sicurezza, controllando il traffico tra due o più reti, con funzioni quali:

- Consentire solo il traffico espressamente autorizzato dalla politica di sicurezza.
- Rilevare e segnalare i tentativi di violazione della sicurezza.
- Eseguire funzioni di audit e accounting (registrazione e controllo degli accessi).
- Collegare segmenti di rete al livello data-link o network.

Tipologie di Firewall

1. Packet Filter (solitamente stateless):

Opera al livello 2 (data-link) o livello 3 (network) del modello OSI. Controlla i pacchetti analizzando gli header IP e TCP, come indirizzi IP e porte TCP. Non tiene traccia dello stato della connessione, quindi valuta ogni pacchetto singolarmente. Router, switch e firewall supportano liste di regole di filtraggio (o controlli di accesso). Le regole vengono controllate sequenzialmente su ogni pacchetto, fino a trovare la prima regola che combacia (se presente). Ogni regola di filtraggio può specificare un'azione di permesso (allow) oppure di negazione (deny/drop).

Anti-spoofing filtering

L'anti-spoofing filtering è una tecnica di filtraggio dei pacchetti usata per bloccare pacchetti con indirizzi IP di origine falsificati (spoofed).

2. Application Gateway (solitamente stateful):

Noto anche come proxy firewall, è un tipo di firewall che opera al livello applicazione (livello 7 del modello OSI). È in grado di monitorare il contenuto semantico dei dati applicativi, anche quando questi si estendono su più pacchetti. Questo consente un'analisi approfondita e contestuale del traffico, rendendo possibile bloccare comandi pericolosi o contenuti non autorizzati anche all'interno di flussi apparentemente leciti. Essendo stateful, il proxy tiene

traccia dello stato delle connessioni, il che permette un controllo più granulare e una gestione più sicura delle comunicazioni tra client e server. In pratica, il traffico verso l'applicazione non arriva direttamente al server, ma viene prima intercettato dal firewall, che agisce da intermediario, controllando, filtrando e inoltrando i dati in base alle politiche di sicurezza definite. Ogni proxy è in genere specializzato per una specifica applicazione: ad esempio, un proxy FTP può essere configurato per permettere solo i download e bloccare gli upload, mentre un proxy e-mail può integrare filtri anti-SPAM per rilevare e bloccare messaggi indesiderati.

Soluzione mista: Screened Subnet:

Una soluzione mista per la sicurezza di rete è rappresentata dalla cosiddetta screened subnet. In questa architettura, il firewall principale accetta connessioni solo verso alcuni servizi esposti su porte note. Quando una connessione in ingresso viene accettata, il firewall esegue una D-NAT, ovvero traduce l'indirizzo di destinazione pubblico in un indirizzo interno, instradando il traffico verso un application gateway dedicato. A questo punto, l'application gateway prende il controllo del traffico, comportandosi da proxy e applica le regole di sicurezza, ispeziona i dati e decide se inoltrare o bloccare la comunicazione, garantendo così un ulteriore livello di protezione tra la rete esterna e i sistemi interni.

Soluzione mista: DMZ (Zona Demilitarizzata)

La DMZ (Demilitarized Zone) è una zona di rete isolata che ospita i servizi esposti pubblicamente, come server web, e-mail o DNS. L'obiettivo è quello di separare la rete interna (protetta) dai servizi accessibili dall'esterno, limitando i rischi in caso di compromissione. In questa configurazione, gli utenti interni e i servizi esposti si trovano su segmenti di rete distinti, ciascuno con livelli di accesso e protezione differenti. Il firewall controlla e filtra il traffico tra:

- Rete esterna → DMZ
- DMZ ← Rete interna

Protocolli di rete con crittografia

TLS: (Transport Layer Security)

TLS è il protocollo di sicurezza che ha sostituito SSL (Secure Sockets Layer), oggi deprecato. Il suo scopo è garantire privacy e integrità dei dati trasmessi tra applicazioni attraverso una rete basata su TCP/IP, come ad esempio il Web.

TLS garantisce la privacy e l'integrità dei dati applicativi scambiati su TCP/IP. Opera al di sopra del livello di trasporto, crittografando i dati dell'applicazione. Il protocollo si compone di due sotto-protocolli principali:

- **Handshake Protocol:** Il protocollo di handshake serve per stabilire una connessione autenticata e protetta tra client e server. È il primo passo necessario per instaurare un canale sicuro, sul quale verrà poi trasmesso il traffico cifrato. Durante questa fase, il client e il server si scambiano i messaggi ClientHello e ServerHello, che contengono informazioni essenziali per sincronizzarsi sulle modalità di comunicazione. In particolare, vengono condivisi:
 - La versione del protocollo TLS da utilizzare
 - Un Session ID, utile per l'eventuale ripresa della sessione in futuro

- Un numero casuale generato da ciascuna parte, utilizzato nella derivazione delle chiavi
- L'elenco delle suite crittografiche supportate dal client, tra cui il server seleziona quella da utilizzare

Successivamente:

Il server invia il proprio certificato digitale X.509, che serve per autenticarsi e permettere al client di verificarne l'identità. Questo certificato contiene anche la chiave pubblica del server. Infine, se il server richiede anche l'autenticazione del client, invia un messaggio di Certificate Request, chiedendo al client di presentare un proprio certificato digitale. Il client invia il proprio certificato digitale X.509 per autenticarsi a sua volta. Subito dopo, il client procede allo scambio della chiave di sessione generando un numero casuale e criptandolo con la chiave pubblica del server in modo che solo il server, con la sua chiave privata, potrà decifrarlo. Infine, TLS prevede due messaggi fondamentali per concludere l'handshake:

- **Change_Cipher_Spec:** Questo messaggio serve a confermare che la suite crittografica è stata impostata correttamente. Da questo momento in poi, tutte le comunicazioni successive saranno cifrate utilizzando la master key e gli algoritmi scelti.
- **Encrypted handshake:** È un messaggio cifrato che contiene l'hash di tutti i messaggi scambiati fino a quel momento nell'handshake. Serve per verificare che l'intero processo non sia stato manomesso, garantendo così l'integrità dell'handshake stesso.
- **Application Data Protocol:** Una volta che l'handshake TLS si è concluso con successo, le applicazioni possono iniziare a comunicare tra loro, scambiando dati in formato cifrato. Da questo momento, tutte le informazioni trasmesse sono protette da confidenzialità, integrità e autenticità, garantite dal canale sicuro stabilito tramite TLS.

Limiti di TSL

Le intestazioni TCP e IP non sono cifrate, quindi gli indirizzi IP e numeri di porta sono visibili. Un attaccante può osservare il flusso del traffico (chi comunica con chi, quanto e quando). Alcune vulnerabilità potenziali nell'utilizzo di SSL/TLS sono lo spoofing degli indirizzi IP e l'analisi del traffico per dedurre comportamenti o informazioni sensibili anche senza accedere ai dati cifrati.

IPsec (Internet Protocol Security)

IPsec è un protocollo di sicurezza che opera a livello IP. La sua configurazione è di tipo host-to-host, risultando trasparente per le applicazioni. Fornisce protezione tra host, ma non specificamente tra le singole applicazioni in esecuzione su di essi. Uno degli utilizzi principali di IPsec è la creazione di comunicazioni tramite VPN.

Il funzionamento di IPsec si basa sull'incapsulamento dei dati. IPsec prende l'intero pacchetto dati originale, che include l'header IP, l'header TCP e i dati dell'applicazione, e lo incapsula in un formato crittografato. A questo pacchetto crittografato viene aggiunto un nuovo header IPsec.

Tunneling: Il tunneling viene utilizzato per creare un “**tunnel**”, gli header IPsec contengono gli indirizzi di due router che agiscono come "endpoint del tunnel".

Il router iniziale (endpoint di partenza) crittografa i pacchetti e li reindirizza verso l'altro endpoint.

Il router finale (endpoint di ricezione) riceve i pacchetti, li decifra e li inoltra alle loro destinazioni reali.

VPN (Virtual Private Network)

Le connessioni VPN sono una comune applicazione della tecnologia IPsec. Vengono spesso impiegate per collegare un dipendente che lavora da remoto con la rete della propria azienda, oppure per connettere tra loro diverse reti locali di una stessa compagnia. All'interno di una connessione VPN, tutto il traffico che transita tra i due endpoint è crittografato.

Difese contro attacchi di rete

IPsec offre una maggiore resistenza a sniffing e modifica dei dati. L'utilizzo di IPsec congiuntamente altri protocolli di crittografia, ad esempio TLS, permette di proteggere le informazioni dei protocolli superiori e quindi di avere più livelli di crittografia e un conseguente aumento di difficoltà nel decifrare. IPsec offre una maggiore protezione contro traffic analysis utilizzando la crittografia a livello di rete per tutti i messaggi, lo scopo applicativo di una comunicazione diventa indistinguibile. IPsec offre anche una maggiore protezione contro IP spoofing autenticando anche l'indirizzo IP d'origine

Sicurezza nelle applicazioni

Lo scopo della sicurezza nelle applicazioni è proteggere le funzionalità e le risorse da accessi non autorizzati, negazione del servizio o utilizzo improprio. Le applicazioni moderne, essendo connesse in rete, sono sempre più esposte e accessibili, rendendole un bersaglio comune per gli attacchi.

Per comprendere la sicurezza, è fondamentale conoscere alcuni termini chiave:

- **Weakness (Debolezza):** Un errore nel progetto o nell'implementazione che può creare una vulnerabilità.
- **Vulnerability (Vulnerabilità):** Un'istanza di una debolezza in un sistema software che può essere sfruttata per violare la policy di sicurezza.
- **Exploit:** Una tecnica usata per sfruttare una vulnerabilità.
- **Attack (Attacco):** Un'azione intenzionale che viola la policy di sicurezza di un sistema.

Attacchi di tipo injection

Gli attacchi basati su "injection" derivano da una debolezza fondamentale: la **neutralizzazione impropria** (CWE-74). Questa si verifica quando un'applicazione non controlla o "purifica" correttamente gli input ricevuti da un utente. Un input manipolato può così penetrare nel sistema ed essere interpretato come un comando o codice, portando a comportamenti imprevisti e dannosi. È essenziale che i controlli sugli input vengano eseguiti sia lato client che, soprattutto, **lato server**, poiché un utente malintenzionato può facilmente aggirare i controlli lato client creando richieste HTTP dirette.

Schema d'attacco XSS (Cross-Site Scripting)

Lo schema di attacco Cross-Site Scripting (XSS) è una forma di code injection. Si verifica quando un utente malintenzionato riesce a inserire ed eseguire codice malevolo, tipicamente scritto in un linguaggio di scripting come Javascript, all'interno del browser di un utente ignaro. Questo sfruttando una vulnerabilità presente in un sito web, che funge da veicolo inconsapevole per l'attacco.

La vulnerabilità di base che permette l'XSS è la **mancaanza di un controllo adeguato (neutralizzazione) sugli input forniti dall'utente**. Se un input non viene correttamente validato o "neutralizzato" prima di essere utilizzato, può essere interpretato in modi imprevisti dall'applicazione, specialmente quando inserito in determinati contesti. È importante notare che i controlli effettuati solo lato client (ad esempio, nella pagina web stessa tramite Javascript) sono insufficienti, poiché un utente malintenzionato può bypassarli inviando richieste HTTP create ad hoc direttamente al server.

Motivazioni e Conseguenze dell'Attacco XSS

Nonostante il codice Javascript malevolo venga eseguito in un ambiente protetto (il browser) che limita la modifica diretta del sistema operativo dell'utente, lo script ha comunque accesso a funzionalità e dati significativi attraverso il browser stesso. Le principali conseguenze di un attacco XSS includono:

- **Furto di cookie:** Lo script può accedere ai cookie della sessione dell'utente (tramite `document.cookie`), inviarli a un server controllato dall'attaccante e utilizzarli per impersonare l'utente o estrarre dati sensibili come gli ID di sessione.
- **Keylogging:** L'attaccante può registrare un "keyboard event listener" (ad esempio, tramite `addEventListener`) per catturare tutti i caratteri digitati dall'utente (incluse potenzialmente password) e inviarli a un proprio server.
- **Phishing:** Lo script può modificare dinamicamente il contenuto HTML della pagina (utilizzando funzionalità del DOM) per creare moduli di login falsi e ingannare l'utente inducendolo a inserire le proprie credenziali.
- **Invio di richieste HTTP arbitrarie:** Lo script può inviare richieste HTTP contenenti qualsiasi dato verso qualsiasi destinazione.

Esistono tre tipi principali di attacchi XSS:

1. **Persistent XSS:** L'attaccante inserisce il codice malevolo (payload) in un campo di input di un sito web (ad esempio, un commento, un post su un forum) che viene poi memorizzato nel database del sito. Quando una vittima richiede la pagina contenente tale dato, il server web, ignaro, include il codice malevolo nella risposta HTML inviata al browser della vittima. Il browser della vittima esegue quindi lo script malevolo. La vulnerabilità risiede nello script lato server che non sanifica l'input prima di memorizzarlo o di includerlo nelle pagine.
2. **Reflected XSS:** L'attaccante crea un URL che include il codice malevolo come parametro (ad esempio, in una query di ricerca) e lo invia alla vittima (tramite e-mail, messaggio, ecc.). Se la vittima clicca sull'URL, il browser invia la richiesta al sito web. Il sito web, se vulnerabile, riflette l'input malevolo (la stringa contenente lo script) direttamente nella pagina HTML di risposta, senza una corretta sanificazione. Ad esempio, un motore di ricerca potrebbe mostrare la keyword ricercata nella pagina dei risultati. Il browser della vittima esegue quindi lo script malevolo contenuto nella risposta. In questo caso, il codice malevolo risiede nella richiesta HTTP della vittima e la vulnerabilità è nello script lato server che gestisce tale richiesta.
3. **DOM-based XSS:** Simile al Reflected XSS, l'attaccante crea un URL con codice malevolo e lo invia alla vittima. Quando la vittima clicca sull'URL, il server web invia una pagina dinamica che, inizialmente, potrebbe non contenere il codice malevolo. Tuttavia, la pagina include codice Javascript (fornito dal server, quindi la vulnerabilità ha origine lì) che è vulnerabile.

Questo script lato client estrae il codice malevolo dall'URL (ad esempio, dalla parte `location.hash` o dai parametri) e lo aggiunge dinamicamente alla pagina HTML (manipolando il DOM, Document Object Model). Il browser interpreta quindi lo script malevolo inserito nel DOM. La vulnerabilità risiede nel codice di script eseguito dal client, sebbene tale codice sia fornito dal server

Contromisure contro XSS

Le contromisure si basano principalmente sull'effettuare operazioni di **purificazione (sanificazione) dell'input utente**. Queste operazioni devono essere configurate tenendo conto di:

- **Contesto:** Il tipo di attacco e la soluzione dipendono da dove l'input utente viene inserito nel codice HTML (ad esempio, contenuto di un elemento, valore di un attributo, URL, valore CSS, valore Javascript). Ad esempio, impedire il carattere " può essere utile se l'input va nel valore di un attributo, ma inutile in altri contesti.
- **Momento del controllo:**
 - **Inbound:** Il controllo avviene quando l'applicazione riceve l'input dall'utente. È più semplice se il sistema ha un unico punto di ingresso.
 - **Outbound:** Il controllo avviene quando l'applicazione invia l'output all'utente. È più semplice se l'output è costruito tramite diverse chiamate.
- **Locazione del controllo:**
 - **Controllo lato server:** Protegge da Persistent e Reflected XSS.
 - **Controllo lato client:** Protegge da DOM-based XSS.

È fondamentale verificare la correttezza degli input utente **sia lato client che lato server**, specialmente in applicazioni client/server (non solo web).

I principali metodi di purificazione sono:

- **Encoding (Codifica):** Consiste nel trasformare (escaping) gli input in modo che il browser non li interpreti come codice eseguibile. Un esempio è l'**HTML escaping**, che converte caratteri speciali come `<` e `>` rispettivamente in `<` e `>`. Questo viene solitamente fatto tramite librerie apposite.
- **Validation (Convalida dell'input):** Si basa su strategie di classificazione per identificare input validi e non validi, e su un outcome che definisce come comportarsi con input non validati. Le strategie di classificazione comprendono:
 - **Blacklisting:** Uno o più pattern (espressioni regolari) identificano gli input pericolosi. È complesso da mantenere perché richiede molti pattern e invecchia velocemente.
 - **Whitelisting:** I pattern identificano gli input validi. La complessità dipende dal tipo di input, ma è generalmente più longevo del blacklisting.

Schema d'attacco SQL Injection

L'SQL Injection è un attacco di tipo **command injection** il cui obiettivo è modificare una query SQL eseguita da un'applicazione, inserendo codice SQL tramite l'input dell'utente. Un attacco riuscito può permettere a un aggressore di leggere o modificare dati sensibili dal database, eseguire comandi amministrativi (come shutdown) o persino accedere a file sul server del database.

La vulnerabilità nasce quando l'input dell'utente viene concatenato direttamente in una stringa di query. La **contromisura** principale, disponibile nella maggior parte dei linguaggi e delle librerie di accesso ai dati, è l'uso di **query parametriche** (o PreparedStatement in Java). Con questo approccio, la query SQL viene precompilata con segnaposto (?), e i valori dell'utente vengono passati separatamente. La libreria si occupa di effettuare l'encoding corretto, garantendo che l'input venga trattato solo come un valore e mai come codice eseguibile.

Attacchi basati su buffer manipulation

Un buffer è uno spazio di memoria, tipicamente un array, dove un'applicazione salva dei dati. In linguaggi come C, non vengono eseguiti controlli per verificare che le operazioni di lettura o scrittura rimangano entro i limiti del buffer. Questo può portare a due tipi di vulnerabilità:

- **Buffer overflow:** si verifica quando si scrive oltre i limiti di un buffer.
- **Buffer overread:** si ha quando si legge oltre i limiti di un buffer.

Queste vulnerabilità sono spesso causate da debolezze nel codice, come la CWE-119, che si manifesta quando il programma accede a un buffer con un indice fuori dal range consentito. Un'altra debolezza comune è la CWE-130, che si verifica quando un parametro che indica la dimensione dei dati di input non corrisponde alla dimensione effettiva dei dati, portando a una lettura o scrittura fuori dai limiti quando i dati vengono trasferiti nel buffer.

Un esempio classico di vulnerabilità è l'uso della funzione `gets()` in C, che legge l'input senza controllare che le dimensioni siano compatibili con quelle del buffer di destinazione. Per questo motivo, è una funzione deprecata. La funzione `fgets()`, considerata più sicura perché permette di specificare una dimensione massima, può comunque introdurre vulnerabilità se il valore massimo fornito è maggiore della dimensione reale del buffer, come nel caso di variabili istanziate con valori non aggiornati dopo una modifica alla dimensione del buffer. La vulnerabilità può anche essere introdotta se la dimensione dell'input viene fornita dall'utente e il programma non verifica che sia inferiore alla capacità del buffer (CWE-130).

Buffer overflow

Quando si verifica un buffer overflow, i dati in eccesso sovrascrivono l'area di memoria adiacente al buffer. Questo è particolarmente pericoloso quando il buffer si trova nello stack, all'interno del frame di una funzione, che contiene le variabili locali, i parametri della funzione, il base pointer del chiamante e l'indirizzo di ritorno.

Un attaccante può sfruttare un buffer overflow per:

- **Modificare variabili locali:** Scrivendo oltre i limiti dell'array, è possibile alterare il valore di altre variabili locali che si trovano nello stack "sopra" il buffer.
- **Dirottare il flusso di esecuzione:** Un attaccante può sovrascrivere l'indirizzo di ritorno della funzione con l'indirizzo di un codice malevolo (shellcode) precedentemente iniettato nel buffer stesso. Al termine della funzione, invece di tornare al chiamante, il programma salterà ed eseguirà lo shellcode.

Lo **shellcode** è spesso un codice che avvia una shell di sistema. Se l'applicazione vulnerabile viene eseguita con privilegi elevati (es. superuser), l'attaccante otterrà una shell con gli stessi privilegi. Per far sì che l'attacco funzioni, l'attaccante deve anche sovrascrivere il base pointer con un valore valido per evitare un crash del programma e calcolare l'indirizzo esatto dello shellcode da inserire come nuovo indirizzo di ritorno.

Esistono anche attacchi di tipo **Heap buffer overflow**, che colpiscono buffer allocati nello heap. Sebbene non possano sovrascrivere l'indirizzo di ritorno, possono alterare altri dati importanti, come i puntatori a funzione.

Contromisure:

- **Canary Values:** Il compilatore inserisce un valore casuale (canary) sullo stack tra le variabili locali e l'indirizzo di ritorno. Prima che la funzione termini, questo valore viene controllato: se è stato modificato (a causa di un overflow), il programma viene terminato. Questa protezione può essere aggirata se l'attaccante riesce a leggere il valore del canary tramite un buffer overread. GCC offre opzioni come `-fstack-protector` per implementare questa misura.
- **Address Space Layout Randomization (ASLR):** Una misura del sistema operativo che randomizza gli indirizzi di memoria di stack, heap e librerie a ogni esecuzione. Questo rende difficile per l'attaccante prevedere l'indirizzo dello shellcode o del base pointer.
- **Executable Space Protection (ESP):** Una protezione hardware e software che marca le aree di memoria contenenti dati (come lo stack e lo heap) come non eseguibili. In questo modo, anche se un attaccante riesce a dirottare il flusso di esecuzione verso il proprio shellcode, la CPU si rifiuterà di eseguirlo. Questa protezione è nota anche come bit NX (No-eXecute) o XD (eXecute Disable).
- **JIT spraying.** È una tecnica di attacco che sfrutta la presenza di compilatori just-in-time (JIT), utilizzati dai runtime di alcuni linguaggi come Java e .NET, per generare codice eseguibile direttamente in linguaggio macchina. Poiché questi runtime sono spesso complessi, possono contenere vulnerabilità che gli attaccanti possono sfruttare. In particolare, il codice generato dal compilatore JIT deve essere collocato in pagine di memoria eseguibili. Gli attaccanti possono quindi usare il compilatore JIT, ad esempio tramite script JavaScript, per generare codice malevolo e poi impiegare vulnerabilità come il buffer overflow o use-after-free per deviare il flusso di esecuzione verso il codice iniettato, eludendo così le protezioni come l'executable space protection.

Buffer overread

Un buffer overread si verifica quando un'applicazione legge dati oltre i limiti di un buffer, potendo così accedere a informazioni sensibili memorizzate in aree di memoria adiacenti.

Un esempio famoso è la vulnerabilità **Heartbleed**. Questa sfrutta una debolezza nell'estensione Heartbeat del protocollo SSL/TLS. L'attaccante invia un messaggio Heartbeat con un payload molto piccolo (es. 1 byte) ma dichiara una lunghezza molto grande (es. 65535 byte). Il server, senza verificare la coerenza tra la lunghezza dichiarata e quella effettiva del payload, legge la lunghezza dichiarata e copia dal proprio buffer di memoria un numero di byte corrispondente (fino a 64KB), che include il piccolo payload e i dati sensibili che si trovano subito dopo in memoria, inviando il tutto all'attaccante.